

LIST DECODING

SAYAN KAR

Last updated on
March 15, 2026

Contents

Notations	iv
1 List Decoding	1
1.1 Introduction	1
1.2 Johnson Bound	2
1.3 List Decoding Capacity	6
1.4 List Decoding from Random Errors	8
1.5 Conclusion	8
2 Decoding of Reed-Solomon Codes	9
2.1 Introduction	9
2.2 Unique Decoding of Reed-Solomon codes	9
2.3 List-Decoding of Reed-Solomon Codes & A Basic Algorithm	11
2.4 Optimizing the List-Decoding Algorithm	13
2.5 Conclusion	15
3 Folded Reed-Solomon Codes	17

Notations

- We denote a finite set of symbols, called an *alphabet*, by Σ . A code $\mathcal{C}$ of block length n over the alphabet Σ is just a subset of Σ^n .
- For $n \in \mathbb{N} := \{1, 2, \dots\}$, we define $[n]$ to be the set $\{1, 2, \dots, n\}$.
- We denote the q -entropy function by $H_q(\rho) := \rho \log_q(q-1) - p \log_q p - (1-p) \log_q(1-p)$.
- Let q be a prime power. Then $\mathbb{F}_q$ denotes the finite field of order q .
- We denote the Hamming distance by $\Delta(\mathbf{x}, \mathbf{y}) := \#\{i \in [n] \mid x_i \neq y_i\}$ for $\mathbf{x}, \mathbf{y} \in \Sigma^n$, and the Hamming Ball by $B(\mathbf{y}, e) := \{x \in \Sigma^n \mid \Delta(\mathbf{x}, \mathbf{y}) \leq e\}$.
- For a bivariate polynomial $Q(X, Y)$, the maximum degree of X in any monomial of $Q(X, Y)$ is denoted by $\deg_X(Q)$ and similarly for Y , we have $\deg_Y(Q)$.

Chapter 1

List Decoding

1.1 Introduction

First we define what is a list decodable codes.

Definition 1.1.1. Given $0 \leq \rho < 1$ and $L \geq 1$, a code $\mathcal{C}$ of block length n over the alphabet Σ is said to be (ρ, L) -list decodable if for all $\mathbf{y} \in \Sigma^n$,

$$\#\{\mathbf{c} \in \mathcal{C} | \Delta(\mathbf{y}, \mathbf{c}) \leq \rho n\} \leq L.$$

Here this ρ parameter is called *fraction of error* and L is called the *list size upper bound*. Observe that, if fraction of error is atmost ρ , then the required codeword would be in the list. Also if $\mathcal{C}$ is (ρ, L) -list decodable, then the decoding algorithm outputs atmost L many codewords as outputs.

Note that the case of $L = 1$ is precisely the scenario of unique decoding, so it would be better to consider $L > 1$. Also the case $L = |\Sigma|^n$ make the notion of list decoding trivial. So we need to be careful about the choice of L .

One way to restrict L is to set to be an arbitrarily large constant. Now we can ask the following question.

Question 1.1.2. What is the limit of ρ as $L \rightarrow \infty$ such that all sufficiently long codes $\mathcal{C} \in \mathfrak{C}$ are (ρ, L) -list decodable ?

We refer to this as the list-decodability of $\mathfrak{C}$ with constant-sized lists. We can also see at a weaker limit keeping our algorithmic goal in consideration. For an efficient algorithm, L has to be a polynomial in n , as otherwise the output size would be super polynomial, giving us an algorithm not having a polynomial running time. This is the list decodability of $\mathfrak{C}$ with polynomial sized lists. For most of the natural codes, the above two notions are the same. For the rest of our discussion we shall only be interested in list decodability with polynomial sized lists.

One of the most useful gain of List Decoding over Unique Decoding is that it generalizes Unique Decoding, as when number of errors is at most half the distance, the list size would be at most one. We shall also see in this section later that for random codes, for most of the error patterns, the list size is at most one with high probability.

Limitations of List Decoding

Since List decoding is a further generalization of Unique Decoding, it is natural to ask the followig two questions, which we shall answer later in this section.

Question 1.1.3. Given a code of relative distance δ , can we correct more than $\frac{\delta}{2}$ fraction of errors using list decoding?

Question 1.1.4. Given a code of rate R , what is the maximum fraction of errors correctable using list decoding?

1.2 Johnson Bound

Theorem 1.2.1. *If $\mathcal{C} \subseteq [q]^n$ is a code of distance d and $\rho < J_q(\frac{d}{n})$, then $\mathcal{C}$ is (ρ, qnd) -list decodable, where the function $J_q(\delta)$ is defined as*

$$J_q(\delta) = \left(1 - \frac{1}{q}\right) \left(1 - \sqrt{1 - \frac{q\delta}{q-1}}\right).$$

(The following proof is for the case of binary alphabet.)

Proof. Denote the error $e = \rho n$. We have to study the code $\mathcal{C} \subseteq \{0, 1\}^n$ with distance d and show that for all $\mathbf{y} \in \{0, 1\}^n$,

$$|B(\mathbf{y}, e) \cap \mathcal{C}| \leq 2dn.$$

Fix a code $\mathcal{C}$ and $\mathbf{y} \in \{0, 1\}^n$ and suppose $B(\mathbf{y}, e) \cap \mathcal{C} = \{\mathbf{c}_1, \dots, \mathbf{c}_M\}$. So we need to show $M \leq 2nd$. Define $\mathbf{c}'_i = \mathbf{c}_i - \mathbf{y}$ for all $i \in [M]$. Consider the sum

$$S = \sum_{i < j} \Delta(\mathbf{c}'_i, \mathbf{c}'_j).$$

As the code is of distance d , then $\Delta(\mathbf{c}'_i, \mathbf{c}'_j) \geq d$ for all $i \neq j$, thus we get

$$S \geq \binom{M}{2} d. \tag{1.1}$$

Consider the $n \times M$ matrix $\mathbf{A} = [\mathbf{c}'_1 : \dots : \mathbf{c}'_M]$. Define m_i to be the number of 1's in the i -th row of $\mathbf{A}$. Observe that the i -th row contributes $m_i(M - m_i)$ to the sum S . Thus

$S = \sum_i m_i(M - m_i)$. Define $\bar{e} := \frac{1}{M} \sum m_i$. Note that

$$\sum_{i=1}^n m_i = \sum_{j=1}^M wt(\mathbf{c}'_j) \leq eM$$

as $wt(\mathbf{c}'_j) = \Delta(\mathbf{c}_j, \mathbf{y}) \leq e$. Thus $\bar{e} \leq e$. Also by RMS-AM inequality,

$$\begin{aligned} S &= \sum_{i=1}^n m_i(M - m_i) = \bar{e}M^2 - \sum_{i=1}^n m_i^2 \\ &\leq \bar{e}M^2 - \frac{(\bar{e}M)^2}{n} \\ &= M^2\left(\bar{e} - \frac{\bar{e}^2}{n}\right). \end{aligned}$$

So we get our combined inequality from the above inequality and (1.1),

$$\binom{M}{2} \leq M^2\left(\bar{e} - \frac{\bar{e}^2}{n}\right).$$

This gives us an upper bound on M , which is the following,

$$\begin{aligned} M &\leq \frac{dn}{dn - 2n\bar{e} + 2\bar{e}^2} = \frac{2dn}{2dn - n^2 + n^2 - 4n\bar{e} + 4\bar{e}^2} = \frac{2dn}{(n - 2\bar{e})^2 - n(n - 2d)} \\ &\leq \frac{2dn}{(n - 2e)^2 - n(n - 2d)} \quad \text{as } \bar{e} \leq e. \end{aligned}$$

We also have the condition $\frac{e}{n} < J_2(\frac{d}{n})$, thus

$$\frac{e}{n} < \frac{1}{2} \left(1 - \sqrt{1 - \frac{2d}{n}}\right) \Rightarrow n - 2e > \sqrt{n(n - 2d)} \Rightarrow (n - 2e)^2 - n(n - 2d) > 0.$$

As all the terms there are natural numbers then, $(n - 2e)^2 - n(n - 2d) \geq 1$. Hence $M \leq 2dn$. □

This alone does not answer the Question 1.1.3. Theorem 1.2.1 along with Lemma 1.2.2 gives us an affirmative answer to the Question 1.1.3.

Lemma 1.2.2. *Let $q \geq 2$ be an integer and $0 \leq x \leq 1 - \frac{1}{q}$, then the following inequality holds.*

$$J_q(x) \geq 1 - \sqrt{1 - x} \geq \frac{x}{2}.$$

The second inequality is strict when $x \neq 0$.

Proof. Consider the function $f(x) = J_q(x) - 1 + \sqrt{1-x}$. The derivative of this function is

$$f'(x) = \frac{1}{2} \left(\frac{1}{\sqrt{1 - \frac{qx}{q-1}}} - \frac{1}{\sqrt{1-x}} \right) \geq 0 \quad \text{for all } x.$$

So the function is increasing and $f(0) = 0$, so we get our first inequality. The second inequality is immediate from the inequality $(1 - \frac{x}{2})^2 \geq 1 - x$. $\square$

Theorem 1.2.1 and Lemma 1.2.2 imply that for a code of relative distance $\delta > 0$, we correct strictly larger fraction of errors than unique decoding in polynomial time as long as q is at most some polynomial in n . These two propositions together also implies an *alphabet free* version of the Johnson Bound.

Theorem 1.2.3 (Alphabet-Free Johnson Bound.). *For any q -ary code $\mathcal{C}$ of block length n and distance d , if $e \leq n - \sqrt{n(n-d)}$, then $\mathcal{C}$ is $(e/n, qnd)$ -list decodable.*

Here I present a Graph Theoretic proof for this theorem with a better list size.

Proof. First we prove the following lemma. Let $G = (L, R, E)$ be a bipartite graph with $|L| = l$ and $|R| = r \geq 2$. For any $s \leq l$, we say G is $K_{s,2}$ -free if there is no subset $L' \subseteq L$ and $R' \subseteq R$ with $|L'| = s$ and $|R'| = 2$ such that $L' \times R' \subseteq E$.

Lemma 1.2.4. *If G is $K_{s,2}$ -free, then $|E| \leq l + r\sqrt{l(s-1)}$.*

Proof. Note that if $|E| \leq l$, then the inequality is trivial. So let us assume that $|E| > l$. Let M be the adjacency matrix of G with order $l \times r$ and M_w be the w -th column of M for $w \in R$. Define

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_l \end{bmatrix} = \sum_{w \in R} M_w$$

Note that v_i denotes the size of the neighbourhood set of the i -th left vertex, and thus $\sum_i v_i = |E|$. By *Cauchy-Schwarz Inequality*,

$$l \cdot \|\mathbf{v}\|_2^2 \geq \left(\sum_i v_i \right)^2 = |E|^2. \quad (1.2)$$

Now take any $S \subseteq L$ of size s and any two vertices $r_1, r_2 \in R$. As G is $K_{s,2}$ -free, $|N_{r_1} \cap N_{r_2}| \leq s - 1$, where $N_w \subseteq L$ denotes the neighbourhood set of $w \in R$. Thus

$$\sum_{r_1 \neq r_2 \in R} |N_{r_1} \cap N_{r_2}| \leq 2 \cdot \binom{r}{2} \cdot (s - 1) = r(r - 1)(s - 1). \quad (1.3)$$

Notice that in the sum in (1.3), the i -th vertex contributes exactly how many pairs (un-ordered) of neighbourhoods there are in L from R so that they both contain that vertex. And the number of such pairs is $2\binom{v_i}{2} = v_i^2 - v_i$. So from the inequality of (1.2), we get:

$$\sum_{i=1}^l v_i^2 - v_i \leq r(r-1)(s-1)$$

$$\Rightarrow \sum v_i^2 \leq \sum v_i + r(r-1)(s-1) \quad (1.4)$$

$$= |E| + r(r-1)(s-1). \quad (1.5)$$

Consider

$$\begin{aligned} (|E| - l)^2 &= |E|^2 + l^2 - 2l|E| \\ &\leq l \cdot \|\mathbf{v}\|_2^2 + l^2 - 2l|E| \quad \text{from (1.2)} \\ &\leq l|E| + lr(r-1)(s-1) + l^2 - 2l|E| \quad \text{from (1.5)} \\ &= l^2 - l|E| + lr^2(s-1) \\ &\leq lr^2(s-1) \quad \text{as } |E| \geq l. \end{aligned}$$

Hence we get our desired result $|E| \leq l + r\sqrt{(s-1)l}$. $\square$

Let $\mathbf{y} \in \Sigma^n$ and $\mathbf{c}^1, \dots, \mathbf{c}^L$ be all the distinct codewords in $\mathcal{C}$ so that $\Delta(\mathbf{y}, \mathbf{c}^i) \leq n - \sqrt{n(n-d)} - 1$ for all $i \in [L]$. Define $G = ([n], [L], E)$ so that $(i, j) \in E$ iff $\mathbf{y}_i = (\mathbf{c}^j)_i$. Next we shall prove that $|E| \geq L + L\sqrt{n(n-d)}$. In the right vertex set, let $\deg(\mathbf{c}^j)$ denote the degree of the j -th right vertex. Observe that $|E| = \sum_j \deg(\mathbf{c}^j)$. Also $\deg(\mathbf{c}^j)$ denotes the number of places where $\mathbf{c}^j$ matches with $\mathbf{y}$, thus $\deg(\mathbf{c}^j) = n - \Delta(\mathbf{c}^j, \mathbf{y})$.

$$\begin{aligned} |E| &= \sum_{j=1}^L \deg(\mathbf{c}^j) \\ &= \sum_{j=1}^L n - \Delta(\mathbf{c}^j, \mathbf{y}) \\ &\geq \sum_{j=1}^L n - (n - \sqrt{n(n-d)} - 1) \\ &= L \cdot \left(1 + \sqrt{n(n-d)}\right). \end{aligned}$$

Observe that if there was a $T \subseteq [n]$ with $|T| = n - d + 1$ and a $S \subseteq [L]$ with $|S| = 2$ so that $T \times S \subseteq E$, then there would be two codewords $\mathbf{c}^{s_1}$ and $\mathbf{c}^{s_2}$ so that they match in at least $n - d + 1$ many places, then $\Delta(\mathbf{c}^{s_1}, \mathbf{c}^{s_2}) \leq d - 1$, which contradicts the fact that $\mathcal{C}$ has

distance d . So G is $K_{n-d+1,2}$ -free. Now from Lemma 1.2.4, $|E| \leq n + L\sqrt{n(n-d)}$. Hence

$$\begin{aligned} L \left(1 + \sqrt{n(n-d)} \right) &\leq n + L\sqrt{n(n-d)} \\ \Rightarrow L &\leq n. \end{aligned}$$

Hence $\mathcal{C}$ is $(1 - \sqrt{1-d/n} - 1/n, n)$ -list decodable. $\square$

Now a natural question is

Question 1.2.5. Is Johnson bound tight? Is it tight for linear codes?

We shall see there are codes, even linear codes, with relative distance δ so that we can find Hamming ball of radius greater than $J_q(\delta)$ with super polynomial many codewords. But we shall also see that, in some sense, it is not tight.

1.3 List Decoding Capacity

Theorem 1.3.1. For $q \geq 2, 0 \leq \rho < 1 - \frac{1}{q}$ and $\varepsilon > 0$ small enough, there exist codes of block length n large enough so that

1. If $R \leq 1 - H_q(\rho) - \varepsilon$, then there is a $(\rho, O(\frac{1}{\varepsilon}))$ -list decodable code.
2. If $R \geq 1 - H_q(\rho) + \varepsilon$, then every (ρ, L) -list decodable code has $L \geq q^{\Omega(\varepsilon n)}$.

Hence capacity of list decoding is $1 - H_q(\rho)$. This is the exact capacity of a qSC_ρ channel. Hence list decoding is the bridge between Hamming and Shannon.

Proof of Theorem 1.3.1

Rather than proving Theorem 1.3.1 directly we prove the following stronger theorem. In fact, we shall show that a random code is (ρ, L) -list decodable with high probability as long as

$$1 - H_q(\rho) - \frac{1}{L}.$$

We shall show that probability of *bad event* is small. *Bad Event* means there exist $\mathbf{m}_0, \dots, \mathbf{m}_L \in [q]^{Rn}$ and $\mathbf{y} \in [q]^n$ so that

$$\Delta(\mathbf{C}(\mathbf{m}_i), \mathbf{y}) \leq \rho n \quad \text{for all } i \in \{0, \dots, L\}.$$

Theorem 1.3.2 (List-Decoding Capacity). Let $q \geq 2$ be an integer, $0 < \rho < 1 - \frac{1}{q}$ and $\varepsilon > 0$.

1. Let $L \in \mathbb{N}$, then there exists a (ρ, L) -list decodable code with rate

$$R \leq 1 - H_q(\rho) - \frac{1}{L}.$$

2. For all (ρ, L) -list decodable code with rate $R \geq 1 - H_q(\rho) + \varepsilon$, it is necessary that $L \geq q^{\Omega(\varepsilon n)}$.

Proof. The following proof is done by Probabilistic Methods.

Proof of 1. Given a received word $\mathbf{y} \in [q]^n$, and $\mathbf{m}_0, \dots, \mathbf{m}_L \in [q]^k$, the tuple $(\mathbf{y}, \mathbf{m}_0, \dots, \mathbf{m}_L)$ defines a *bad event* if

$$C(\mathbf{m}_i) \in B(\mathbf{y}, \rho n) \quad \text{for all } i \in \{0, \dots, L\}.$$

Note that $\mathcal{C}$ is (ρ, L) -list decodable if there is no *bad event*.

Fix $\mathbf{y} \in [q]^n$, and $\mathbf{m}_0, \dots, \mathbf{m}_L \in [q]^k$. Note that for fixed i , by the choice of $\mathcal{C}$, we have

$$\Pr[C(\mathbf{m}_i) \in B(\mathbf{y}, \rho n)] = \frac{\text{Vol}_q(\rho n, n)}{q^n} \leq q^{-n(1-H_q(\rho))}.$$

The probability of the *bad event* by $(\mathbf{y}, \mathbf{m}_0, \dots, \mathbf{m}_L)$ is

$$\begin{aligned} \Pr \left[\bigwedge_{i=0}^L C(\mathbf{m}_i) \in B(\mathbf{y}, \rho n) \right] &= \prod_{i=0}^L \Pr[C(\mathbf{m}_i) \in B(\mathbf{y}, \rho n)] \quad [\text{Independence of } C(\mathbf{m}_i)\text{'s}] \\ &\leq q^{-n(L+1)(1-H_q(\rho))}. \end{aligned}$$

Now we need to have no *bad events* for $\mathcal{C}$ in order to have it (ρ, L) -list decodable.

$$\begin{aligned} \Pr[\text{There is no bad event}] &\leq q^n \binom{q^k}{L+1} q^{-n(L+1)(1-H_q(\rho))} \\ &\leq q^n q^{k(L+1)} q^{-n(L+1)(1-H_q(\rho))} \quad \left[\binom{n}{k} \leq n^k \right] \\ &= q^{-n(L+1)[1-H_q(\rho) - \frac{1}{L+1} - R]} \quad [R = kn] \\ &\leq q^{-n(L+1)[1-H_q(\rho) - \frac{1}{L+1} - 1 + H_q(\rho) + \frac{1}{L}]} \\ &= q^{-\frac{n}{L}} \\ &< 1. \end{aligned}$$

The first inequality is apparent from going through all the received words and messages possible and using the union bound. Thus, by probabilistic method, there exists a code $\mathcal{C}$ which is (ρ, L) -list decodable. Also the argument tells us that probability of a random code not being (ρ, L) -list decodable goes to 0 as $n \rightarrow \infty$.

Proof of 2 We need to show that there exists $\mathbf{y} \in [q]^n$ so that $|C \cap B(\mathbf{y}, \rho n)|$ is exponentially large for every $\mathcal{C}$ of rate $R \geq 1 - H_q(\rho) + \varepsilon$. Pick $\mathbf{y} \in [q]^n$ uniformly at random and fix a

$\mathbf{c} \in \mathcal{C}$. Then

$$\begin{aligned} \Pr[\mathbf{c} \in B(\mathbf{y}, \rho n)] &= \Pr[\mathbf{y} \in B(\mathbf{c}, \rho n)] \\ &= \frac{\text{Vol}_q(\rho n, n)}{q^n} \quad [\text{as } \mathbf{y} \text{ is uniformly random}] \\ &\geq q^{-n(1-H_q(\rho))-o(n)}. \end{aligned}$$

We have

$$\begin{aligned} \mathbb{E}[|\mathcal{C} \cap B(\mathbf{y}, \rho n)|] &= \sum_{\mathbf{c} \in \mathcal{C}} \mathbb{E}[\mathbb{1}_{\mathbf{c} \in B(\mathbf{y}, \rho n)}] \\ &= \sum_{\mathbf{c} \in \mathcal{C}} \Pr[\mathbf{c} \in B(\mathbf{y}, \rho n)] \\ &\geq |\mathcal{C}| \cdot q^{-n(1-H_q(\rho))-o(n)} \\ &= q^{n[R-1+H_q(\rho)-o(1)]} \\ &\geq q^{n(\epsilon-o(1))} \\ &\geq q^{\Omega(\epsilon n)}. \end{aligned}$$

Hence there exists $\mathbf{y}$ so that $|\mathcal{C} \cap B(\mathbf{y}, \rho n)| \geq q^{\Omega(\epsilon n)}$. $\square$

Finally, we have shown that list decoding capacity is $1 - H_q(\rho)$. However the proof does not give us any explicit code that can achieve this capacity. Thus we must ask the following questions.

Question 1.3.3. Do there exist explicit codes that achieve that achieve list decoding capacity?

Question 1.3.4. Can we achieve list decoding capacity with efficient list decoding algorithm?

Question 1.3.5. Can we design an efficient algorithm that can achieve the Johnson Bound?

1.4 List Decoding from Random Errors

1.5 Conclusion

We got a brief introduction to the notion of List Decoding and proved that it generalizes Unique Decoding and also some limitations of List Decoding. Although we didn't see any explicit codes and decoding algorithms, we proved their existence probabilistically and shall see those explicit examples in the coming sections through decoding of Reed-Solomon Codes.

Chapter 2

Decoding of Reed-Solomon Codes

2.1 Introduction

Having been built an introduction to the concept of *list-decoding*, we shall see its implementation on a “good code”, namely the Reed Solomon Code. But first we shall the *Unique Decoding* of Reed-Solomon Code, then jump into the list decoding setup. Consider the $[n, k, d = n - k + 1]_q$ RS code over the field $\mathbb{F}_q$ with the evaluation points $(\alpha_1, \dots, \alpha_n) \in \mathbb{F}_q^n$.

2.2 Unique Decoding of Reed-Solomon codes

The problem of RS code unique decoding is the following:

Problem 2.2.1 (Reed-Solomon Unique Decoding).

- (Input) $\mathbb{F}_q, (\alpha_1, \dots, \alpha_n) \in \mathbb{F}_q^n, k$ and recieved word $\mathbf{y} \in \mathbb{F}_q^n$.
- (Output) $P(X) \in \mathbb{F}_q[X]$ of degree less than or equal to $k - 1$ such that $e = \Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) < (n - k + 1)/2$, if such polynomial exists or return fail.

Before going into the actual algorithm for this problem, we need to define a few things and also some ideas to prove the correctness of this algorithm.

Definition 2.2.2 (Error-Locator Polynomial). For a transmitted polynomial $P(X)$, a non-zero polynomial $E(X)$ is called an *error-locator polynomial* if for all $i \in [n]$, we have:

$$E(\alpha_i) = 0 \text{ if } y_i \neq P(\alpha_i).$$

One possible candidate error-locator polynomial is:

$$E(X) = \prod_{\substack{i \in [n] \\ y_i \neq P(\alpha_i)}} (X - \alpha_i).$$

Also note that for all $i \in [n]$,

$$y_i E(\alpha_i) = E(\alpha_i) P(\alpha_i). \quad (2.1)$$

Here both $E(X)$ and $P(X)$ are unknown and we want to find out $P(X)$ for this problem. Also it would suffice if we knew $E(X)$ only, but to solve the n constraints in (2.1), we need to find solution to these n quadratic equations with the variables being the coefficients of $E(X)$ and $P(X)$. This is a hard problem to do, thus we linearize this to solve this particular problem in polynomial time. To perform this linearization, define $N(X) = P(X)E(X)$ and notice that $N(X)$ has degree less than or equal to $e + k - 1$, so if we know $N(X)$ and $E(X)$, then we are done. Algorithm 2.2 guarantees to give such $N(X)$ and $E(X)$ in polynomial time.

Algorithm 1 Welch-Berlekamp Algorithm

Input: $n \geq k \geq 1$, $0 < e < (n - k + 1)/2$ and n pairs $\{(\alpha_i), y_i\}_{i=1}^n$ with α_i 's distinct.

Output: Polynomial $P(X)$ of degree at most $k - 1$ or fail.

```

1  Compute a non-zero polynomial  $E(X)$  of degree  $e$ , and a polynomial  $N(X)$  of degree at
   most  $e + k - 1$  such that  $N(\alpha_i) = y_i E(\alpha_i)$  for all  $i \in [n]$ .
2  if  $E(X)$  and  $N(X)$  as above do not exist or  $E(X)$  does not divide  $N(X)$  then
3      return fail
4  end if
5   $P(X) \leftarrow \frac{N(X)}{E(X)}$ .
6  if  $\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) > e$  then
7      return fail
8  else
9      return  $P(X)$ 
10 end if
```

Correctness of Algorithm 2.2

Theorem 2.2.3. *If $(P(\alpha_i))_{i=1}^n$ is transmitted, with $\deg P \leq k - 1$ and at most $e < (n - k + 1)/2$ errors occur, then Welch-Berlekamp Algorithm outputs $P(X)$.*

Before going to the proof of this theorem, notice that this algorithm uniquely decodes the RS code of rate R for up to $(1-R)/2$ fraction of errors.

Claim. *There exists polynomials $E(X)$ and $N(X)$ over $\mathbb{F}_q$ satisfying Step 1 such that $\frac{N(X)}{E(X)} = P(X)$.*

Proof. Taking $E(X)$ to be an error-locating polynomial of $P(X)$, we define $N(X) = E(X)P(X)$. To be precise, we are taking $E(X)$ to be the polynomial

$$E(X) = X^{e - \Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n)} \prod_{\substack{i \in [n] \\ y_i \neq P(\alpha_i)}} (X - \alpha_i)$$

so that $\deg(E(X)) = e$ and thus $\deg(N(X)) \leq e + k - 1$. Also by just the definition of $E(X), N(X)$, we can conclude that $N(\alpha_i) = y_i E(\alpha_i)$ for all $i \in [n]$. $\square$

So we have proved that such a pair of polynomials exists, now the main challenges lies for the Step 4 as we might have a pair (E', N') satisfying Step 1 but $\frac{N'}{E'} \neq P$. Though the next claim solidifies the fact that we shall have the same polynomial from any pair (E, N) after the execution of Step 1,2 and 3.

Claim. *If any two distinct solutions (E_1, N_1) and (E_2, N_2) satisfy Step 1, then we have:*

$$\frac{N_1}{E_1} = \frac{N_2}{E_2}.$$

Proof. We define the polynomial $R(X)$ like following:

$$R(X) = N_1(X)E_2(X) - N_2(X)E_1(X).$$

Observe that $\deg(R) \leq 2e + k - 1 < n$, by the condition on e . But for all $i \in [n]$,

$$R(\alpha_i) = y_i E_1(\alpha_i) E_2(\alpha_i) - y_i E_1(\alpha_i) E_2(\alpha_i) = 0.$$

Hence R has to be the zero polynomial, thus we are done. $\square$

Proof of Theorem 2.2. From Claim 2.2, we ensure that a solution (E, N) exists such that $N(X)/E(X) = P(X)$, thus the algorithm outputs a pair (E', N') satisfying Step 1 and by Claim 2.2, we ensure that $\frac{N'}{E'} = P$. $\square$

2.3 List-Decoding of Reed-Solomon Codes & A Basic Algorithm

In Question 1.3.5, we asked if it possible to list decode a code of rate R upto $1 - \sqrt{R}$ fraction of errors, given that it meets the Singleton bound, i.e, $\delta = 1 - R$. The answer to this question is affirmative for Reed-Solomon codes which we shall gradually prove. The setup of the problem for this section is the following:

Problem 2.3.1 (Reed-Solomon List-Decoding).

- (Input) $\mathbb{F}_q, (\alpha_1, \dots, \alpha_n) \in \mathbb{F}_q^n, k$ and recieved word $\mathbf{y} \in \mathbb{F}_q^n$.
- (Output) A list of polynomials over $\mathbb{F}_q$ of degree less than k such that the number of agreement points, i.e., $t := \#\{i \in [n] | y_i = P(\alpha_i)\} \geq n - e$.

Our goal is to minimize t as much as possible and Johnson bound allows us to make t as small as $\sqrt{nk}$, which is ofcourse better than the unique decoding setup by AM-GM inequality, as there t was $(n + k)/2$. We shall modify the *Welch-Berlekamp Algorithm* a bit

to solve the Reed-Solomon list-decoding problem. The main setup of the Welch Berlekamp Algorithm can be given by the two steps as following:

1. (Interpolation) Find a non-zero $Q(X, Y)$ so that for all $i \in [n]$, $Q(\alpha_i, y_i) = 0$.
2. (Root Finding) If $Y - P(X)$ is a factor of $Q(X, Y)$, then output $P(X)$.

The equivalence of Welch-Berlekamp and the above two steps is apparent by taking $Q(X, Y) = YE(X) - N(X)$ and $Y - P(X)$ divides $Q(X, Y)$ if and only if $P(X) = N(X)/E(X)$. For List-Decoding, Step 1 can be taken care of by giving some kind of bound to the degree of $Q(X, Y)$ then applying *Gaussian Elimination* and Step 2 can be taken care of by any fast polynomial factorization algorithm, i.e., can be solved in linear time.

We shall see three algorithms for solving the List-Decoding of RS codes with better amount of fraction of errors to each other in order. First we shall see the Basic List-Decoding with fraction of errors $1 - 2\sqrt{R}$, then Weighted Degree List-Decoding with fraction of errors $1 - \sqrt{2R}$ and finally the Multiplicity List-Decoding with fraction of errors $1 - \sqrt{R}$, hence finally getting to the Johnson Bound.

Algorithm 2 Basic List-Decoding Algorithm for Reed-Solomon Codes

Input: $n \geq k \geq 1, l \geq 1, e = n - t$ and n pairs $\{(\alpha_i, y_i)\}_{i=1}^n$.

Output: A list of polynomials $P(X)$ of degree at most $k - 1$.

- 1 **Step 1.** Compute a non-zero $Q(X, Y) \in \mathbb{F}_q[X, Y]$ with $\deg_X(Q) \leq l, \deg_Y(Q) \leq n/l$ such that

$$Q(\alpha_i, y_i) = 0 \quad \text{for all } i \in [n].$$

- 2 **Step 2.** Factor $Q(X, Y)$ into irreducible factors $Q_1(X, Y), Q_2(X, Y), \dots, Q_m(X, Y)$.
 - 3 $L \leftarrow \phi$.
 - 4 **for** every factor $Q_j(X, Y) = Y - P_j(X)$ of $Q(X, Y)$ **do**
 - 5 **if** $\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) \leq e$ and $\deg(P_j) \leq k - 1$ **then**
 - 6 $L \leftarrow P_j(X)$.
 - 7 **end if**
 - 8 **end for**
 - 9 **return** L .
-

Proof of Correctness of Algorithm 2.3

First we need to show that the coefficients of $Q(X, Y)$ are more than the number of constraints, i.e., n , so that we can do the interpolation. Ofcourse this is true as the number of coefficients of are is $(l + 1)(n/l + 1) > n$, so such $Q(X, Y)$ exists.

Now to ensure the root finding step, we need to show that for every polynomial $P(X)$ of degree at most $k - 1$ in the list L at the final step, if agrees with Y in at least t positions, then $Y - P(X)$ divides $Q(X, Y)$. For this, consider the polynomial $R(X) = Q(x, P(X))$ and

$$\deg(R) \leq l + \frac{n(k - 1)}{l}.$$

Observe that $Y - P(X)$ divides $Q(X, Y)$ if and only if $R(X) \equiv 0$. Also $R(X)$ has at least t roots. So for $R(X)$ to be constant 0, we can have $t > \deg(R)$ to get a contradiction via Degree Mantra. Hence,

$$t > l + \frac{n(k-1)}{l}$$

would be helpful for us. If we take $l = \sqrt{n(k-1)}$, we get $t > 2\sqrt{n(k-1)}$. So we can list-decode for up to $1 - 2\sqrt{R}$ fraction of errors. $\square$

2.4 Optimizing the List-Decoding Algorithm

In this section, we shall try to optimize the Basic List-Decoding Algorithm and to get past that $1 - 2\sqrt{R}$ fraction of errors bound. First we shall see an algorithm using weighted degrees with which we can list-decode up to $1 - \sqrt{2R}$ fraction of errors and then an algorithm using multiplicities with which we can extend that to $1 - \sqrt{R}$.

Definition 2.4.1. For a bivariate polynomial $Q(X, Y)$, the $(1, w)$ -weighted degree of the monomial $X^i Y^j$ is $i + wj$ and the $(1, w)$ -weighted degree of $Q(X, Y)$ is the maximum $(1, w)$ -weighted degree of all the monomials in $Q(X, Y)$.

We now precisely describe the Weighted Degree List-Decoding Algorithm next.

Algorithm 3 Weighted Degree List-Decoding Algorithm for Reed-Solomon Codes

Input: $n \geq k \geq 1, D \geq 1, e = n - t$ and n pairs $\{(y_i, P(\alpha_i))_{i=1}^n\}$.

Output: A list of polynomials of degree at most $k - 1$.

- 1 Compute a polynomial $Q(X, Y)$ of $(1, k - 1)$ -weighted degree at most D such that $Q(\alpha_i, y_i) = 0$ for all $i \in [n]$.
 - 2 $L \leftarrow \phi$.
 - 3 **for** every factor $Y - P(X)$ of $Q(X, Y)$ **do**
 - 4 **if** $\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) \leq e$ and $\deg(P) \leq k - 1$ **then**
 - 5 $L \leftarrow P(X)$.
 - 6 **end if**
 - 7 **end for**
 - 8 **return** L
-

Note that like in the previous algorithm, we have the degree of $R(X) = Q(X, P(X))$ to be less than or equal to $\deg_X(Q) + (k - 1)\deg_Y(Q) \leq D$, so we need to take $t > D$, thus somehow claim $t > \sqrt{2n(k-1)}$. Also we don't need to bother about the second step because of the same reason as before.

Proof of Correctness of Algorithm 2.4

For the interpolation part, we need to study the total number of coefficients of $Q(X, Y)$. Write Q as the following:

$$Q(X, Y) = \sum_{\substack{0 \leq i, j \leq D \\ i+j \leq D}} c_{ij} X^i Y^j.$$

The total number of coefficients in Q is exactly the sum

$$\sum_{i=0}^D \sum_{j=0}^{D-i} 1 = \sum_{i=0}^D \left(\frac{D-i}{k-1} + 1 \right) \quad (2.2)$$

$$= \frac{2k(D+1) + D^2 - 3D - 2}{2(k-1)} \quad (2.3)$$

$$\geq \frac{D(D+2)}{2(k-1)}. \quad (2.4)$$

Here the inequality in (2.4) can be obtained via some basic algebraic manipulations. Now if we choose $D = \sqrt{2n(k-1)}$, we have this sum to be greater than n . Thus having $t > \sqrt{2n(k-1)}$. $\square$

Theorem 2.4.2 (Weighted Degree List-Decoding). *Algorithm 2.4 can list-decode Reed-Solomon codes of rate R from up to $1 - \sqrt{2R}$ fraction of errors. Further, the algorithm runs in polynomial time with output list size at most $O(1/\sqrt{R})$.*

Note that if the list size is l , then $(k-1) \cdot l$ is less than D , hence l is $O(1/\sqrt{R})$.

Multiplicity List Decoding

We now impose more conditions on the degree of Q by introducing the concept of multiplicity.

Definition 2.4.3. We say a bivariate polynomial $Q(X, Y) \in \mathbb{F}_q[X, Y]$ has r zeroes at $(0, 0)$ for some $r \in \mathbb{N}$ if there is no monomial in Q of degree less than or equal to $r-1$ and for $(a, b) \in \mathbb{F}_q^2$, we say Q has r zeroes at $(a, b) \in \mathbb{F}_q^2$ if $Q_{a,b}(X, Y) := Q(X+a, Y+b)$ has r zeroes at $(0, 0)$.

Proof of Correctness of Algorithm 2.4

Lemma 2.4.4. *For every $(\alpha_1, \dots, \alpha_n) \in \mathbb{F}_q^n$ and every $r \in \mathbb{N}$, there exists $Q(X, Y) \in \mathbb{F}_q[X, Y]$ of $(1, k-1)$ -weighted degree at most $r\sqrt{(1+1/r)nk}$ such that Q has a zero at (α_i, y_i) of multiplicity r for all $i \in [n]$.*

Proof. We shall prove this claim by proving the following claims.

Algorithm 4 Multiplicity List-Decoding Algorithm for Reed-Solomon Codes**Input:** $n \geq k \geq 1, D \geq 1, r \geq 1, e = n - t$ and n pairs $\{(y_i, P(\alpha_i))_{i=1}^n\}$.**Output:** A list of polynomials of degree at most $k - 1$.

- 1 Compute a polynomial $Q(X, Y)$ of $(1, k - 1)$ -weighted degree at most D such that $Q(\alpha_i, y_i) = 0$ with multiplicity r for all $i \in [n]$.
- 2 $L \leftarrow \phi$.
- 3 **for** every factor $Y - P(X)$ of $Q(X, Y)$ **do**
- 4 **if** $\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) \leq e$ and $\deg(P) \leq k - 1$ **then**
- 5 $L \leftarrow P(X)$.
- 6 **end if**
- 7 **end for**
- 8 **return** L

Claim. For each $i \in [n]$, there are $\binom{r+1}{2}$ constraints on coefficients of Q corresponding to $Q(\alpha_i, y_i) = 0$.

Proof. Let $Q_{a,b}(X, Y) = \sum_{i,j} c_{i,j}^{a,b} X^i Y^j$ with the condition that $0 \leq i, j \leq D$ and $i + (k - 1)j \leq D$ and $Q(X, Y) = \sum_{i',j'} c_{i',j'} X^{i'} Y^{j'}$ with the condition that $0 \leq i', j' \leq D$ and $i' + (k - 1)j' \leq D$. Now we have,

$$\begin{aligned} Q_{a,b}(X, Y) &= \sum_{i,j} c_{i,j}^{a,b} X^i Y^j \\ &= \sum_{i',j'} c_{i',j'} (X + a)^i (Y + b)^j. \end{aligned}$$

Next observe that for all i, j ,

$$c_{i,j}^{a,b} = \sum_{\substack{i' \geq i \\ j' \geq j}} \binom{i'}{i} \binom{j'}{j} c_{i',j'} a^{i'-i} b^{j'-j}.$$

We also have $c_{i,j}^{a,b} = 0$ for all $i + j < r$, thus we get total $n \binom{r+1}{2}$ constraints, $\binom{r+1}{2}$ corresponding to each α_i . $\square$

Now having this claim proved we move onto the proof of multiplicity if the points (α_i, y_i) , i.e. we now show the proof of the following claim.

Claim. $R(X) := Q(X, P(X))$ has α_i as a root with multiplicity r for all $i \in [n]$ such that $y_i = P(\alpha_i)$.

Proof. $\square$

$\square$

2.5 Conclusion

Chapter 3

Folded Reed-Solomon Codes

Bibliography

- [GRS24] V. Guruswami, A. Rudra, and M. Sudan, *Essential Coding Theory*, Graduate Studies in Mathematics, vol. 226, American Mathematical Society, 2024.
- [Sud25] M. Sudan, *Algebra in Algorithmic Coding Theory*, [arXiv:2512.06478](#), 2025.
- [GS99] V. Guruswami and M. Sudan, Improved decoding of Reed–Solomon and algebraic–geometric codes, *IEEE Trans. Inform. Theory* **45** (1999), no. 6, 1757–1767. Preliminary version in *Proc. FOCS 1998*. [doi:10.1109/18.782097](#).